

Enterprise Banking Microservices

A Scalable and Secure Distributed Banking Backend — a project for Database Systems Engineering and Distributed Backend Development.

Snigdha (2520090160),
Koumudi (2520090161),
Sumanjali (2520090190)

ABSTRACT

Enterprise Banking Microservices is a distributed backend system designed to provide scalable, secure, and reliable banking services using a microservice architecture. The system divides major banking operations such as customer management, account management, transactions, payments, and authentication into independent services. Each microservice can operate and scale independently while communicating through well-defined APIs. A distributed database approach is used to maintain data consistency, reliability, and efficient access to banking information. The project demonstrates important database engineering concepts such as transaction management, concurrency control, data integrity, service-level database design, and secure data access. The architecture also supports fault isolation and scalability, making it suitable for handling large numbers of users and banking transactions. Overall, the project provides practical experience in designing and implementing a distributed banking backend using modern microservices and database technologies.

Introduction

Traditional banking systems tend to be large, tightly coupled monoliths that struggle with growth and resilience. Our project applies a microservices architecture so each banking capability is independent: developed, deployed, scaled, and maintained on its own lifecycle.

Primary Goal

Design a scalable, secure, and reliable distributed backend for core banking operations.

Communication

Services interact via well-defined REST APIs and an API Gateway.

Problem Statement

Monolithic banking backends create operational and engineering constraints that impact availability, scalability, and security. Key issues include:

- Hard-to-scale individual operations
- Tight coupling that increases blast radius of failures
- High complexity managing concurrent transactions and data consistency
- Elevated risk for sensitive-data exposure

❏ Our approach: independent microservices + distributed databases + secure APIs to address these problems.

Proposed System – Microservices



Customer Service

Registration, profile management, KYC metadata; owns Customer DB.



Account Service

Account creation, balances, and product metadata; owns Account DB.



Transaction Service

Deposits, withdrawals, transfers – durable transaction records in Transaction DB.



Payment Service

External payments, settlement, and reconciliation; Payment DB.



Authentication Service

Login, JWT issuance, password hashing, MFA hooks; Auth DB isolated for credentials.

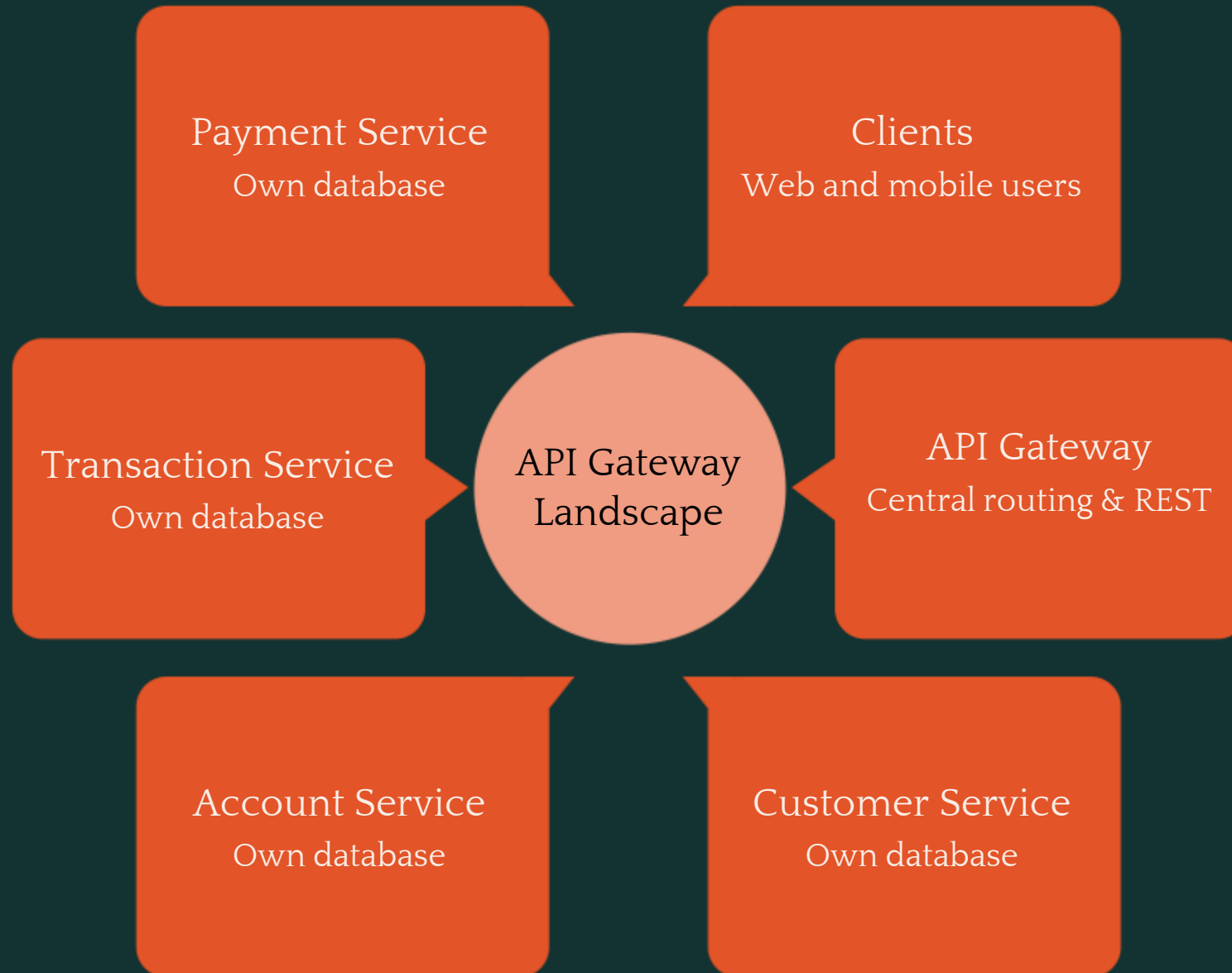
Each service exposes REST endpoints and owns its data to minimize coupling.

Fraud Detection & Risk Analysis

- Independent **Fraud Detection Microservice**
- Analyzes transactions in real time
- Identifies unusual transaction patterns
- Generates a **Risk Score (0–100)**
- **Low Risk → Approve**
- **Medium Risk → Additional Verification**
- **High Risk → Flag / Block**
- Can use **anomaly detection (e.g., DBSCAN)** for suspicious transaction identification

Transaction → Behavioral Features → Anomaly Detection → Risk Score → Decision → Feedback

System Architecture



Database Design — Service-Level Ownership

Each microservice manages the data required for its domain to reduce coupling and allow independent scaling:

- **Customer DB:** profiles, KYC documents, audit logs
- **Account DB:** account metadata, balance snapshots, product links
- **Transaction DB:** immutable ledger entries, timestamps, idempotency keys
- **Payment DB:** settlement records, external payment statuses
- **Auth DB:** credential hashes, refresh tokens, MFA state

Key database engineering concepts applied: transaction management, concurrency control, data integrity, secure access, and eventual consistency patterns where appropriate.



Technology Stack



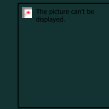
Backend

Spring Boot and Node.js (Express) — Java and JavaScript implementations.



Databases

PostgreSQL / MySQL per service (managed or containerized).



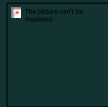
Microservices

Spring Cloud Eureka for discovery; Spring Cloud Gateway as API Gateway.



Security

Spring Security + JWT tokens, HTTPS, token refresh and MFA-ready hooks.



Dev & Testing

Docker, Postman, Swagger/OpenAPI, GitHub, IntelliJ/VS Code for CI and local development.



Security & Reliability Patterns

Security and resilience are first-class concerns:

- Authentication: JWT + refresh tokens, password hashing, optional MFA
- Authorization: role-based scopes enforced at gateway and service level
- Transport: TLS for all inter-service and client traffic
- Resilience: circuit breakers, retries with backoff, bulkheads
- Observability: centralized logging, distributed tracing, metrics and alerts

Advantages & Expected Outcome

Scalability

Scale individual services based on workload (e.g., Transaction service for peak hours).

Fault Isolation

Service failures are contained; degraded service can be routed or throttled without full system outage.

Security & Maintainability

Separated auth domain and secure APIs reduce attack surface; independent deployments simplify maintenance.

Performance

Specialized services and tuned databases deliver efficient handling of diverse workloads.

Expected deliverable: a working prototype demonstrating microservices, distributed data ownership, secure API flows, and end-to-end transactions.

Next Steps & Project Plan

01

Phase 1 — Core Services

Implement Customer, Account, and Auth services with DB schemas and basic REST APIs.

03

Phase 3 — Ops & Hardening

Integrate API Gateway, service discovery, security hardening, observability, and CI/CD pipelines.

02

Phase 2 — Transactions & Payments

Build Transaction and Payment services with idempotency, reconciliation, and durability.

04

Deliverables

Prototype codebase, architecture docs, demo scenarios, and evaluation report for the course.

THANK YOU